

services\cache\cache_manager.py

```
import threading
from datetime import datetime, timezone, timedelta
from typing import Dict, List, Any, Optional

class CacheManager:
    """Lightweight caching system with reduced memory footprint"""

    def __init__(self):
        # Only cache metadata, not actual CVE data
        self._cache_timestamps = {}
        self._historical_loaded = False
        self._lock = threading.Lock()

        # Cache duration configuration (in seconds)
        self.API_CACHE_DURATION = 24 * 60 * 60 # 24 hours
        self.HISTORICAL_CACHE_DURATION = 7 * 24 * 60 * 60 # 7 days

    def get_api_data_30_days(self) -> Optional[List[Dict[str, Any]]]:
        """Check if API data cache is valid - returns None, DB should be checked"""
        cache_key = "api_30_days"
        with self._lock:
            if cache_key in self._cache_timestamps:
                timestamp = self._cache_timestamps.get(cache_key, 0)
                age = datetime.now().timestamp() - timestamp
                if age < self.API_CACHE_DURATION:
                    print(f"[Cache] API cache still valid (age: {int(age/60)} minutes)")
                    return True # Signal to use DB
                return None

    def set_api_data_30_days(self, data: List[Dict[str, Any]]):
        """Set cache timestamp for API data"""
        cache_key = "api_30_days"
        with self._lock:
            self._cache_timestamps[cache_key] = datetime.now().timestamp()
            print(f"[Cache] Set timestamp for API data ({len(data)} CVEs)")

    def get_historical_data(self) -> Optional[Dict[str, List[Dict]]]:
        """Check if historical data is loaded"""
        if self._historical_loaded:
            print("[Cache] Historical data marked as loaded")
            return True
        return None

    def set_historical_data(self, data: Dict[str, List[Dict]]):
        """Mark historical data as loaded"""
        with self._lock:
            self._historical_loaded = True
            total_cves = sum(len(year_data) for year_data in data.values())
            print(f"[Cache] Marked historical data as loaded ({total_cves} CVEs)")

    def get_cache_stats(self) -> Dict[str, Any]:
        """Get cache statistics"""
        with self._lock:
            stats = {
                'cached_timestamps': len(self._cache_timestamps),
                'historical_loaded': self._historical_loaded,
                'cache_ages': {}
            }

        current_time = datetime.now().timestamp()
        for key, timestamp in self._cache_timestamps.items():
            age_minutes = int((current_time - timestamp) / 60)
            stats['cache_ages'][key] = f"{age_minutes} minutes"

        return stats

    def clear_cache(self):
        """Clear all cached data"""
        with self._lock:
            self._cache_timestamps.clear()
            self._historical_loaded = False
            print("[Cache] All caches cleared")
```

```
def warm_up_cache(self):  
    """Warm up cache - does nothing now since we use DB"""  
    print("[Cache] Cache warm-up skipped (using database)")  
  
    # Global cache instance  
    cache_manager = CacheManager()
```